

Computer Networks

“Final Project Report:

ChatFlow: *Encrypted Bluetooth Mesh Chat for Nearby Devices*

Submitted By

Abdul Rahman Azam

23K-0061

Hafsa Rashid

23K-0064

FAST School of Computing

National University of Computer & Emerging Sciences

1. Motivation

In many real-world scenarios-disaster zones, remote areas, or crowded events with overloaded cellular infrastructure-internet access is simply unavailable. Conventional messaging applications cannot function without a server and an active internet connection, leaving nearby people unable to communicate privately. ChatFlow addresses this gap by exploiting Bluetooth Low Energy (BLE), a radio interface built into every modern smartphone, to enable direct device-to-device chat with no internet or infrastructure required. All messages are AES-128 encrypted before transmission, ensuring that even if a relay node can eavesdrop on the wireless channel, message contents remain confidential.

2. Overview

ChatFlow is an Android application for end-to-end encrypted, offline messaging over Bluetooth Low Energy. From a computer-networks perspective it is a wireless ad-hoc mesh: each device simultaneously operates as a BLE peripheral (advertising) and a BLE central (scanning), forming connections with multiple peers. Messages not addressed to the local node are flood-forwarded to all other connected peers, giving the network multi-hop reachability without any central router. Loop prevention is achieved through a UUID-based deduplication cache and a TTL (Time To Live) field that decrements at each hop.

2.1 Significance of the Project

ChatFlow integrates core Computer Networks concepts-wireless link layer, ad-hoc mesh topology, application-layer routing, fragmentation and reassembly, and symmetric encryption-into a single working prototype. It has real-world value in disaster response, remote field operations, or any off-grid scenario where internet connectivity is unavailable or untrusted. The project was implemented entirely with free, open-source Android tooling and demonstrates that infrastructure-free secure communication is achievable on commodity smartphones.

2.2 Description of the Project

ChatFlow is an Android application for offline encrypted messaging using Bluetooth Low Energy. Each device can scan for nearby peers while simultaneously advertising itself for connection. Messages can travel through intermediate connected devices to reach users outside direct Bluetooth range, forming a multi-hop mesh. Large messages are divided into smaller numbered chunks and reassembled at the receiver. All messages are encrypted using AES-128-CBC via the built-in javax.crypto library before transmission. To prevent forwarding loops, every message carries a unique UUID and a TTL value; duplicate messages detected by UUID are silently discarded.

2.3 Background of the Project

Bluetooth Low Energy (BLE) is a wireless communication technology supported on Android devices since API level 21 (Android 5.0). BLE uses GATT (Generic Attribute Profile) Services and Characteristics to exchange data between devices. ChatFlow is inspired by Bluetooth mesh networking principles, where devices forward messages for one another without any central infrastructure [1][2]. AES (Advanced Encryption Standard) is a symmetric block cipher standardised by NIST and is widely used for securing data at rest and in transit [3][5]. Android provides AES support through the `javax.crypto` package without requiring any third-party library. The Bluetooth Core Specification [4] defines the link-layer constraints-MTU, advertising payload limits, and duty-cycle behaviour-that directly shape ChatFlow's fragmentation and discovery design.

2.4 Project Category

ChatFlow is a Product-Based project. It produces a fully functional, installable Android APK that can be deployed on real devices to provide offline encrypted mesh communication. The project is academic in origin but has direct real-world applicability in any environment where internet connectivity is unavailable or untrusted.

3. Features / Scope / Modules

ChatFlow is organised into three modules, each implementing a distinct layer of functionality. The features below are listed from most to least significant.

- **BLE Mesh Discovery and Multi-Device Connection.**

Each device simultaneously advertises as a GATT server and scans for nearby GATT servers, enabling it to maintain concurrent connections to multiple peers. Connection status is tracked in real time and shown in the UI.

- **Flood-Based Mesh Relay with Loop Prevention.**

When a received message is not addressed to the local device, it is forwarded to all other connected peers with TTL decremented. A UUID-keyed deduplication cache (entries expire after 120 s, bounded by LRU eviction) ensures each message is relayed at most once per node, preventing broadcast storms.

- **AES-128-CBC End-to-End Encryption.**

All message payloads are encrypted before BLE transmission and decrypted only after reassembly at the destination, using the standard `javax.crypto` library with no third-party dependency. A fresh 16-byte initialisation vector (IV) generated by `SecureRandom` is included in each message header, so identical plaintexts always produce distinct ciphertexts.

- **Message Fragmentation and Reassembly.**

Messages exceeding the negotiated 512-byte BLE MTU are fragmented after encryption (encrypt-then-fragment). Each chunk carries the message UUID, chunk index, and total chunk count. The

receiver buffers chunks for up to 60 seconds and reassembles out-of-order arrivals before passing the ciphertext to the decryption layer.

- **Application-Layer Message Protocol.**

Each message frame carries: senderId, recipientId, messageUuid, timestamp (Unix ms), ttl, chunkIndex, totalChunks, iv (16 bytes), and the AES ciphertext payload. This protocol is compact enough to fit within BLE MTU constraints while carrying all metadata required for routing, dedup, and decryption.

- **Real-Time Chat UI and Local Chat History.**

The interface displays chat bubbles annotated with sender nickname and timestamp. A Room (SQLite) database persists all conversations across app restarts. A foreground Android Service keeps BLE connections alive and delivers notifications when the app is in the background.

- **Automatic Reconnection.**

On unexpected peer disconnection the BLE manager retries with exponential backoff, recovering from transient radio outages without user intervention.

4. Project Planning

The project was executed over five weeks. The table below details the planned and actual schedule for each task, the responsible team member, and completion status.

Task / Activity	Planned	Actual	Owner	Done
Requirement Analysis, Architecture & UI Mockups	Week 1	Week 1	Both	Yes
BLE Device Manager: Scanning, GATT, Multi-Device	Week 2	Week 2	Abdul Rahman	Yes
Chat Manager: AES Encryption, Framing, Mesh Relay	Week 3	Week 3	Hafsa Rashid	Yes
UI, Chat History, Room DB & Notifications	Week 4	Week 4	Abdul Rahman	Yes
Integration, Multi-Device Testing & Debugging	Week 5	Week 5	Both	Yes
Documentation & Final Report	Week 5	Week 5	Both	Yes

Table 1. Project schedule with planned vs. actual timeline, ownership, and completion status.

5. Project Feasibility

5.1 Technical Feasibility

All required APIs - BLE GATT dual-role operation, javax.crypto AES, and the Room persistence library - are part of the standard Android SDK available from API level 21 onward and require no special hardware beyond a commodity Android smartphone. The dual-role BLE capability (simultaneous peripheral and central) was verified on the test devices before development began. The main technical risk is OEM-specific BLE stack instability, a known variability across Android manufacturers; this is mitigated by the exponential-backoff reconnection policy implemented in the BLE Device Manager module.

5.2 Economic Feasibility

The project has zero monetary cost. Android Studio, Kotlin, and Gradle are free and open-source. No cloud services, paid SDKs, server infrastructure, or specialised hardware are required beyond off-the-shelf Android smartphones already owned by the team.

5.3 Schedule Feasibility

Five weeks was well suited to the scope of the project. The modular decomposition (Device Manager, Chat Manager, UI) allowed both team members to work independently on well-defined interfaces, minimising integration friction. All milestones were completed on schedule as shown in Table 1.

6. Hardware and Software Requirements

6.1 Hardware

- **Android Devices:** Minimum two Android smartphones running Android 5.0 (API 21) or above, with BLE hardware supporting simultaneous peripheral and central roles. A third device is required for multi-hop mesh relay testing.
- **Development Machine:** Any laptop or desktop capable of running Android Studio.

6.2 Software

- **Android Studio (Hedgehog or later):** Primary IDE and build environment.
- **Kotlin 1.9:** Implementation language.
- **Android SDK (API 21+):** BLE GATT APIs and standard cryptography libraries.
- **Room Persistence Library:** SQLite database abstraction for chat history storage.
- **javax.crypto (standard Android SDK):** AES-128-CBC encryption and decryption; no additional library required.

7. Diagrammatic Representation of the Overall System

The following screenshots are from the working application demonstrating the complete user flow from startup through to active encrypted chat sessions.

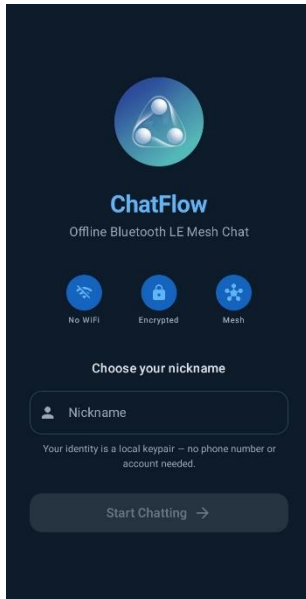


Figure 1: App startup with nickname entry.

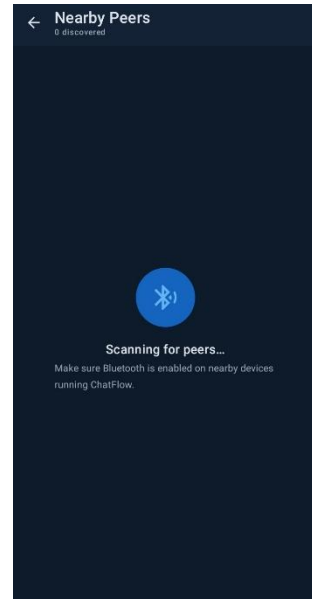
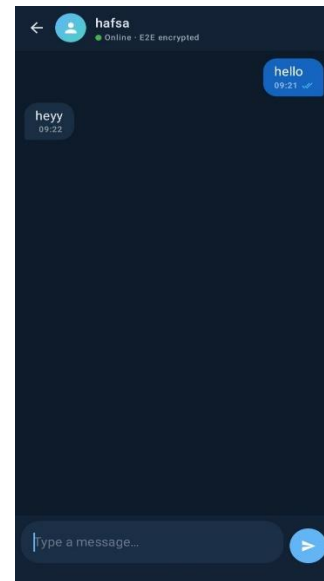


Figure 2: BLE scanning for nearby peers.



Figures 3-4: Peer discovered and active E2E encrypted chat

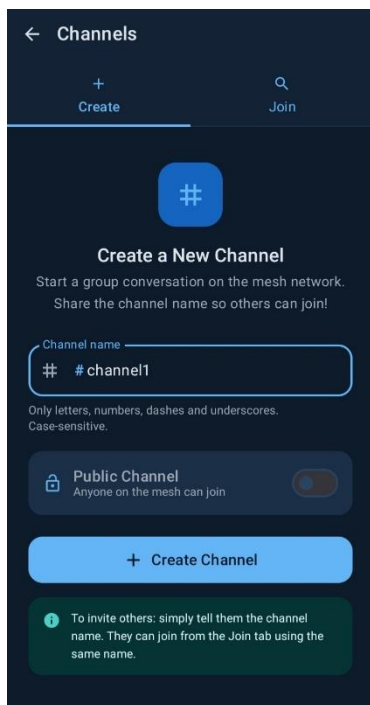


Figure 5: Creating a new named channel

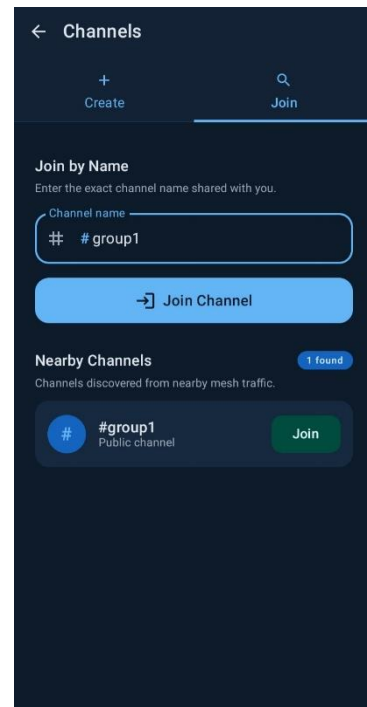


Figure 7: Searching and joining for channel

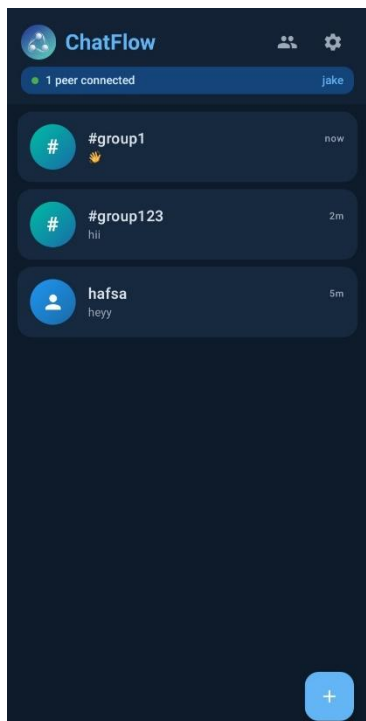


Figure 8: Main sessions view showing all active chats and connected peers.

8. References

1. Android Bluetooth Guide – Official Android documentation

<https://developer.android.com/guide/topics/connectivity/bluetooth>

2. Bluetooth Low Energy (BLE) Overview – Official Android documentation

<https://developer.android.com/guide/topics/connectivity/bluetooth-le>

3. AES Encryption in Java/Kotlin – Oracle official cryptography guide

<https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html>